

Sistemas Operativos
3 de febrero de 2021, Curso 2020-21
Facultad de Informática, UCM

Cuestión 1. El sistema de ficheros de un computador sigue un esquema basado en FAT-32 (utilizando por tanto direcciones de bloque con tamaño de 4 bytes). Además, se sabe que el tamaño de un directorio siempre es un múltiplo exacto del tamaño de bloque. En un determinado momento, la tabla FAT contiene la siguiente información:

0	1	2	3	4	5	6	7	8	9	10	11	12	13
EOF	EOF	10	30	44	13	34	21	22	23	1	32	40	20
14	15	16	17	18	19	20	21	22	23	24	25	26	27
28	27	6	33	26	11	35	38	36	3	50	5	52	48
28	29	30	31	32	33	34	35	36	37	38	39	40	41
45	16	EOF	EOF	EOF	9	12	42	47	4	31	19	EOF	55
42	43	44	45	46	47	48	49	50	51	52	53	54	55
51	25	53	37	43	18	24	46	41	8	39	EOF	2	29

En este sistema, la ruta de un fichero es /SO/Exámenes/Febrero2021.docx. De los directorios y el fichero se conocen los siguientes datos:

	Primer bloque	Tamaño	Tipo
SO	17	192 bytes	D
Exámenes	7	128 bytes	D
Febrero2021.docx	54	100 bytes	F

- a) Represente esquemáticamente la misma situación de disco, pero suponiendo que, en vez de FAT, se dispone de un sistema de ficheros tipo UNIX. En este sistema, la estructura del inodo está compuesta por tres punteros directos, un puntero indirecto simple y un puntero indirecto doble.
- b) Indique y explique el número máximo y mínimo de accesos a bloques de disco que se requieren para leer todo el fichero Febrero2021.docx utilizando:
- a) El sistema de asignación basado en FAT descrito inicialmente. Supóngase que el primer bloque del directorio SO es conocido, y no hay ningún bloque de la FAT cargado inicialmente en memoria.
- a) El sistema de asignación tipo UNIX indicado. Supóngase que el inodo de SO se encuentra cargado en memoria y que cada inodo ocupa exactamente un bloque de disco.

Cuestión 2. Se desea simular el trabajo en un muelle de carga, usando hilos para simular los operarios que realizan la carga y los camiones que son cargados. El siguiente código describe la estructura de los hilos utilizados para los trabajadores y los camiones:

```
#define NP 10 // Capacidad máxima de cada camión

void *worker(void *arg)
{
    while (1) {
        // esperar a poder entrar

        enter_dock();

        // esperar a que el camión esté cargado

        delivery();
    }
}

void *truck(void *arg)
{
    while (1) {
        // esperar a poder cargar

        load_parcel(p);

        // notificar si cargado
    }
}
```

Los operarios (`worker`) repiten indefinidamente las acciones de coger un paquete (`get_parcel`) y cargarlo en el camión (`load_parcel`). Varios operarios pueden estar cogiendo un paquete al mismo tiempo, pero cargarán el camión de uno en uno. El hilo del operario sólo debe ejecutar la función `load_parcel` cuando haya algún camión en el muelle y quede sitio en dicho camión para el paquete.

Los camiones (`truck`) tiene capacidad para cargar `NP` paquetes, y repiten indefinidamente las acciones de entrar al muelle (`enter_dock`) para recoger la carga y salir al reparto (`delivery`). Varios hilos de camión pueden estar ejecutando la función `delivery` al mismo tiempo. Sin embargo, en el muelle sólo cabe un camión y por tanto el hilo que simula al camión debe esperar a que el muelle quede libre para ejecutar la función `enter_dock` en exclusiva. Asimismo, el hilo debe esperar a que el camión esté completamente cargado para salir del muelle y empezar el reparto.

Completa las funciones `truck` y `worker` para que los hilos se sincronicen correctamente, añadiendo las variables que consideres necesarias.

Solución:

```
#define N 10
#define M 3
#define NP 10

mutex_t mtx;
cond_t space, loaded, dempty;
int dock_empty = 1;
int truck_space;

void *truck(void *arg)
{
    int id = (int) arg;

    while (1) {
        mutex_lock(&mtx);
        while (!dock_empty)
            cond_wait(&dempty, &mtx);
        truck_space = NP;
        dock_empty = 0;
        cond_broadcast(&space);

        enter_dock(id);

        while (truck_space)
            cond_wait(&loaded, &mtx);
        dock_empty = 1;
        cond_signal(&dempty);
        mutex_unlock(&mtx);

        delivery();
    }
}

void *worker(void *arg)
{
    int id = (int) arg;
    int p;

    while (1) {
        p = get_parcel();

        lock(&mtx);
        while (dock_empty || truck_space < 1)
            cond_wait(&space, &mtx);
        truck_space--;

        load_parcel(id, p);

        if (truck_space == 0)
            cond_broadcast(&loaded);
        unlock(&mtx);
    }
}
```

Cuestión 3. Un SO aplica una política de planificación con colas multinivel (3 niveles) y realimentación a las tareas siguiendo las siguientes directrices:

- Las tareas nuevas entran al nivel 1.
- En todos los niveles se aplica planificación **Round-Robin**, con cuantos crecientes: $q=2$ (nivel1), $q=3$ (nivel2), $q=4$ (nivel3).
- Las tareas del nivel 1 que durante una ráfaga de CPU agoten su cuanto y necesiten más CPU pasarán al nivel 2. Si la tarea finaliza su ráfaga justo al agotar su cuanto permanece en el nivel.
- Las tareas del nivel 2 que durante una ráfaga de CPU agoten su cuanto y necesiten más CPU pasarán al nivel 3. Si la tarea finaliza su ráfaga justo al agotar su cuanto permanece en el nivel.
- Cada 10 unidades de tiempo, comenzando en el instante 0, (instantes 10,20,...) todos los procesos que se encuentren en el tercer nivel, o estén bloqueados por entrada salida pero fueron planificados por última vez desde este nivel, recibirán una actualización de su prioridad y pasarán a la cola de nivel 1. Esta mejora es independiente del instante del instante de llega a dicho nivel. Si hay varias tareas que mejoran su prioridad, el orden que tuvieran entre ellas en el nivel 3 se conserva en el nivel 1.
- Las tareas que retornen de entrada salida se mantendrán en el nivel en el que fueron planificadas por última vez, independientemente del nivel.

A) Identificas problemas que puede presentar este algoritmo de planificación?. En caso afirmativo, describirlos razonando la respuesta.

B) Aplíquelo a las siguientes trazas:

Tarea	Llegada	CPU	E/S	CPU	E/S	CPU
T1	0	1	3	1	4	2
T2	1	3	1	2	3	1
T3	2	6	1	1		
T4	3	6	1	1		

Completar el diagrama temporal, indicando también los procesos preparados que hay en cada una de las colas. Al rellenar el diagrama temporal, se ha de usar el siguiente convenio:

- En ejecución: marcar con X
- Bloqueado por E/S: marcar con O
- Listo para ejecutar: marcar con –

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
T1																								
T2																								
T3																								
T4																								
Q1																								
Q2																								
Q3																								

Cuestión 4. Un sistema utiliza gestión de memoria virtual con paginación bajo demanda con páginas de 4KiB, direcciones de memoria de 64 bits y memoria principal de 16GB. En dicho sistema se desea ejecutar el siguiente programa:

```
#include <stdio.h>
/* resto de includes */
...

#define N 2048
int A[N] = {0};
int *B;

void* copyv(void* arg);

int main(int argc, char* argv[])
{
    int pid, i;
    pthread_t th1, th2;
    int arg1[2], arg2[2];

    // Punto A
    for (i=0; i<N; i++)
        A[i] = i+1;
    // Punto B
    printf("Initialization was OK");
    // Punto C
    B=malloc(N*sizeof(int))
    // Punto D
    arg1[0] = 0;    arg1[1] = N/2 - 1;
    arg2[0] = N/2; arg2[1] = N-1;
```

```
    pthread_create(&th1, NULL,
        copyv, (void*) arg1);
    pthread_create(&th2, NULL,
        copyv, (void*) arg2);
    // Punto E
    pthread_join(th1, NULL);
    pthread_join(th2, NULL);
    // Punto F
    for (i=0; i<N; i++)
        printf("B[%i]=%i\n", i, B[i]);
    // Punto G
    return 0;
}

void* copyv(void* arg)
{
    int* params = (int*) arg;
    int i;
    for (i=params[0]; i <= params[1]; i++)
        B[i] = A[i];

    return NULL;
}
```

Considerando que:

- Las llamadas al sistema y el resto de funciones de librería que se incluyen en el programa nunca fallan.
- Hay suficientes marcos de página libres en el sistema.
- Todas las funciones externas se incluyen en una única biblioteca cuyo texto ocupa 2,7 KiB, y su sección de datos (todos inicializados) ocupa 1,3 KB. Dicha biblioteca se enlazó de forma dinámica al crear el ejecutable.
- El texto (código) del programa tras la compilación ocupa 3,5KiB. Dicha región de texto incluye el código para la carga dinámica de bibliotecas, y se ha de suponer que la región de datos inicializados de la biblioteca se modifica durante su carga, ya que ésta incluye un código de inicialización.
- El contenido inicial de la pila al lanzar a ejecución el programa ocupa 128 bytes.
- Un entero en C ocupa 4 bytes.

Responda a las siguientes preguntas:

- a. Identifique qué regiones lógicas (y su tamaño en páginas) constituyen la imagen de memoria del proceso multihilo en los puntos marcados en el código como A, B, C, D, E, F y G.
- b. ¿Cuántos fallos de página se producirán al pasar del punto A al punto B?. ¿Y del punto F al G? Justifique sus respuestas.

Cuestión 5. Un proceso en UNIX ejecuta el siguiente código:

```
1  #include <fcntl.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <sys/types.h>
5  #include <unistd.h>
6  #include <sys/wait.h>
7  #include <pthread.h>
8
9  #define N 5
10 #define MAX_LINE 9
11
12 char message[] = "Hello Student";
13 int fd;
14 int *studentseq;
15 char *color[3]={"BLUE","YELLOW","NONE"};
16
17 void check (int id) {
18
19     char NIF[MAX_LINE];
20
21     fd=open("students.txt", O_RDONLY);
22     int pos= id * (MAX_LINE+1);
23
24     lseek (fd, pos, SEEK_SET);
25     read (fd, NIF, MAX_LINE-1);
26
27     if ( atoi (NIF) % 2 == 0 ){
28         printf ("Blue: %s\n", NIF);
29         studentseq [id]=0;
30     }
31     else {
32         printf ("Yellow: %s\n", NIF);
33         studentseq [id]=1;
34     }
35     close (fd);
36 }
37
38 int main(){
39
40     int i,j;
41     pid_t pid;
42
43     studentseq = (int *)malloc(N * sizeof (int));
44
45     for(i=0;i<N;i++)
46         studentseq [i]=2; //NONE
47
48     for (j = 0; j < N; j++) {
49         pid = fork();
50         if (pid < 0) {
51             perror("fork");
52             exit (EXIT_FAILURE);
53         } else if (pid == 0) {
54             check(j);
55             exit (EXIT_FAILURE);
56         }
57     }
58     while(execlp("/bin/echo", "/bin/echo",
59         message, NULL) > 0);
60     printf ("pid= %d ppid= %d student %d= %s\n",getpid(),
61         getppid(), 3, color [ studentseq [2]]);
62     exit (EXIT_SUCCESS);
63 }
```

Asumiendo que el proceso original tiene PID 100 (y es hijo de *Init*), se pide:

- Explica razonadamente cuántos procesos serán creados y cuantos ejecutarán potencialmente de forma concurrente.
- Justifica si se puede determinar o no el orden en que se ejecutarán estos procesos, indicando en tal caso dicho orden. En cualquier caso, indica una posible salida del código anterior.
- Indica una posible salida si cambiamos la función `execlp (" . . . ")` por `wait (NULL)`.

NOTA: Originalmente el fichero `students.txt` tiene almacenado:

```
63513380D
63513381R
63513382F
63513383X
63513384S
```